

Project Proposal

Summer 2026– DSA 5900– 1 Credit Hour

Danny Nguyen

Faculty Supervisor: Dr. Matthew Beattie

Thesis Advisor: Dr. Charles Nicholson

Contents

List of Tables	i
1 Introduction	1
2 Objectives	2
2.1 Technical Project Objectives	2
2.2 Individual Learning Objectives	2
3 Plan	3
3.1 Data Description and Transformation	3
3.2 Methodological Stages	3
3.2.1 Stage 1: Activation Extraction and Vector Alignment	4
3.2.2 Stage 2: Linear Probing and Parameter Classification	4
3.2.3 Stage 3: Symbolic Mapping and Graph Construction	5
4 Deliverables	6
4.1 Technical Deliverables	6
4.2 Anticipated Visualizations	6
5 Schedule	8
6 References	9

List of Tables

1 Project Milestones and Targeted Completion Dates	8
--	---

1 Introduction

In the modern media landscape, the velocity and volume at which information spreads make manual verification insufficient, leaving professional journalists and publishers working under severe time constraints. This problem has catalyzed a growing interest in fact-checking systems capable of parsing arbitrary text, retrieving evidence, and predicting the veracity of assertions. [2] [3] However, contemporary semantic pipelines frequently rely on generative large language models (LLMs) to output structured configurations like JSON, introducing critical flaws: they are highly prone to hallucination, fragile under slight parsing variances, and computationally expensive at scale. [4] [5] [8]

To overcome these limitations, this practicum aims to define a low-compute, symbolic framework to extract the underlying predicate logic from synthetic natural language reasoning examples using frozen transformer activations. By deploying a compact, open-weights model—Meta’s Llama 3.2-3B—this practicum extracts high-dimensional activation vectors from the model’s intermediate residual stream layers. Rather than relying on text generation, these numerical parameters will feed into shallow, linear classification probes trained to deterministically isolate atomic assertions and identify propositional relationships (e.g., implication, negation, and conjunction) directly from the embeddings within a model’s latent space. [1] [6]

2 Objectives

The overarching goal of this practicum is to develop a robust, parameter-level extraction pipeline that bypasses generative LLM bottlenecks to map natural language into verifiable logical graphs. The specific project and individual objectives are outlined below:

2.1 Technical Project Objectives

- **Latent Space Probing Architecture:** Design and train low-compute, linear classification heads (probes) to extract propositional claims and logical relationships directly from the intermediate residual stream of Meta’s Llama 3.2-3B model.
- **Symbolic Pipeline Integration:** Construct a deterministic pipeline that translates extracted latent representations into discrete symbolic tokens without relying on stochastic text-generation or JSON parsing.
- **Graph-Based Fact Verification:** Implement a visualization and verification layer that structures the extracted symbols into a formal logical graph, for human interpretation of the underlying logical structures of natural language.
- **Performance Optimization:** Evaluate the classifier framework to achieve high factual and structural precision while reducing inference-time computational overhead compared to baseline autoregressive generative prompting.

2.2 Individual Learning Objectives

- **Mechanistic Interpretability Mastery:** Gain deep, practical expertise in analyzing the internal geometry of transformers, specifically concerning layer-wise activation tracking, representation dynamics, and internal knowledge localization.
- **Advanced Representation Engineering:** Master state-of-the-art diagnostic probing and activation patching methodologies using specialized interpretability toolkits.
- **Symbolic Architecture Development:** Develop a rigorous engineering workflow for bridging machine representations (vector embeddings) with classical symbolic paradigms (graph structures).

3 Plan

To achieve the project objectives and transition from text generation to parameter-level classification, this practicum will deploy a structured, multi-stage methodology. The engineering process encompasses dataset preprocessing, model activation extraction, linear probe training, and symbolic graph generation.

3.1 Data Description and Transformation

This project will utilize the **LogicNLI** dataset[7], a diagnostic benchmark explicitly designed to evaluate structural logical reasoning in natural language processing models.

- **Dataset Size and Format:** The dataset consists of 20,000 synthetic text pairs divided into training, validation, and testing splits, formatted as structured parquet entries.
- **Native Fields:** Each native entry contains three fields: **premise** (the context statement), **hypothesis** (the query statement), and **label** (a categorical value denoting whether the hypothesis is entailed by, neutral to, or contradicts the premise).
- **Data Transformation and Feature Engineering:** Because the native dataset lacks explicit labels for isolated logical components, a deterministic preprocessing step will be introduced. Using rule-based semantic parsing (via regular expressions and token-dependency matching), each **premise** string will be programmatically decomposed into:
 1. **Atomic Terms:** Individually isolated propositional variables (e.g., extracting “Alice is in the garden” as Token Block *A*).
 2. **Relational Connectives:** Explicit markers representing the logical syntax joining those terms (e.g., identifying “If... then” structures as implication ($\rightarrow$), “not” as negation ($\neg$), and “and” as conjunction ($\wedge$)).

These transformed, fine-grained structural targets will serve as the ground-truth annotations used to train the relational linear probes.

3.2 Methodological Stages

The technical execution of the project is divided into three major operational stages:

3.2.1 Stage 1: Activation Extraction and Vector Alignment

The first stage involves routing the preprocessed natural language input strings through a frozen Meta Llama 3.2-3B architecture. To accommodate a consumer-grade, low-compute evaluation environment, the execution pipeline will be tested under two hardware paradigms: an enterprise laptop utilizing an Intel Core i7-13620H CPU, and a consumer desktop equipped with an NVIDIA RTX 4070 Ti (12GB VRAM) GPU. To strictly adhere to the VRAM (or lack thereof) and memory constraints of these target systems while maximizing throughput, input sequences will be processed in deterministic batches of size $b = 16$ or $b = 32$, depending on the active hardware configuration.

Rather than running the model to its final token-generation layer, PyTorch forward-hook functions will be injected into the transformer’s intermediate blocks—specifically isolating the middle residual stream layers (layers 8 through 16). At these layers, the forward pass will be intercepted to extract high-dimensional hidden state representations.

To align token spans with our pre-identified atomic propositions, the Hugging Face tokenizer’s `return_offsets_mapping` parameter will be leveraged. This maps the character indices of each rule-parsed proposition directly to its corresponding token position indices in the tensor. The extracted hidden states within these verified token boundaries will then undergo mean-pooling to construct static, $\mathbb{R}^{3072}$ semantic representation vectors for each propositional clause and connective.

3.2.2 Stage 2: Linear Probing and Parameter Classification

Using the extracted intermediate activation vectors as frozen input features, shallow probes will be trained using standard logistic regression and minimal multi-layer perceptrons (MLPs). Separate classification heads will be optimized via cross-entropy loss to predict the structural primitives established during our data transformation stage:

1. The presence and boundaries of distinct atomic propositional terms within the text vector space.
2. The operational connectives joining them (e.g., classifying a latent vector as containing an active negation, conjunction, or implication relation).^[1]

By restricting these heads to linear layers, the pipeline ensures that any successful classification reflects structured, linearly separable logical information natively encoded by the Llama model’s internal geometry.

^[1]In order to maintain an achievable schedule for this practicum, the initial implementation will focus on a limited set of logical relations; starting with implications. Further relations will be added iteratively as time allows.

3.2.3 Stage 3: Symbolic Mapping and Graph Construction

Once the linear probes output deterministic, parameter-driven classifications, these discrete predictions will be mapped into a classical symbolic syntax (e.g., representing the text as nodes A and B connected by a directed edge $\rightarrow$). A visualization and validation script written in NetworkX will intake these extracted symbols to construct a formal logical graph representation. This graph will serve as the auditable interface for automated fact-checking, bypassing generative hallucinations entirely.

4 Deliverables

To fulfill the project objectives and provide the sponsor with a functional, verifiable neuro-symbolic framework, this practicum will yield a suite of open-source software modules, trained artifacts, and interpretive visual assets.

4.1 Technical Deliverables

- **Data Augmentation and Preprocessing Engine:** A Python-based pipeline that ingests raw LogicNLI data and applies deterministic rule-based semantic parsing to output token-aligned ground-truth annotations for atomic propositions and logical connectives.
- **Transformer Activation Extraction Framework:** A PyTorch and Hugging Face-based framework utilizing forward hooks to interface with a frozen Meta Llama 3.2-3B model. This module intercepts the intermediate residual stream (layers 8–16) and extracts aligned, mean-pooled hidden state tensors without triggering token generation.
- **Linear Probe Model Artifacts:** A collection of serialized, low-compute linear classification heads (logistic regression and shallow MLP layers) optimized to map internal vector geometries to discrete symbolic primitives (e.g., terms, negations, implications).
- **Symbolic Logic Graph Compiler:** A software script that converts numerical classification logits into structured symbolic notation and automatically compiles them into a graph database structure using the NetworkX library.
- **Performance Evaluation Suite:** A comprehensive benchmarking suite comparing the parameter-level classification framework against baseline zero-shot generative LLM prompting, measuring metrics such as inference latency, memory consumption, parsing failure rates, and structural accuracy.

4.2 Anticipated Visualizations

To effectively communicate the empirical findings and prove the existence of internal logical geometry, the following visual assets will be generated:

- **Latent Space Dimensionality Reductions (PCA/t-SNE):** Two- and three-dimensional scatter plots visualizing the geometry of the residual stream. These plots will illustrate how vectors representing distinct logical relations (e.g., statements containing negation vs. implication) naturally cluster and linearly separate within Llama’s middle layers.

- **Layer-Wise Accuracy Trajectories:** Line graphs tracking the classification accuracy of the linear probes across every layer of the 3B model. This visualization will map the "semantic engine" of the transformer, demonstrating where abstract logical structures converge in the middle layers and where they dissolve in the late generation layers.
- **Interpretable Symbolic Graph Networks:** Automated graph node-link diagrams rendering the final pipeline output. These will visually represent natural language premises as clear, auditable network graphs where vertices depict extracted atomic terms and directed, color-coded edges depict verified logical operations.

5 Schedule

To ensure the systematic execution and successful completion of this project, a comprehensive work management schedule has been established. The target completion date for this practicum is **August 2026**.

The multi-stage implementation plan is organized into the following consecutive bi-weekly milestones:

Table 1: Project Milestones and Targeted Completion Dates

Milestone	Description of Activities and Key Deliverables	Target
Milestone 1	Data Engineering & Hook Extraction: Process the LogicNLI dataset to extract atomic terms/connective labels, and simultaneously implement the PyTorch forward-hook architecture to extract Llama 3.2-3B intermediate residual stream tensors.	June 19
Milestone 2	Probe Training: Train and cross-validate linear classification heads on the extracted vector representations; optimize for term and relation detection.	July 3
Milestone 3	Graph Compilation: Develop the downstream translation module to map probe logits into discrete network symbolic graph architectures.	July 10
Milestone 4	Evaluation & Visualization: Run comparative benchmarks against generative baselines; generate PCA geometric plots and layer-wise accuracy charts.	July 17
Milestone 5	Final Reporting: Document engineering pipelines, compile project artifacts for the sponsor, and finalize the comprehensive practicum report.	July 24

6 References

- [1] Guillaume Alain and Yoshua Bengio. Understanding intermediate layers using linear classifier probes, 2018.
- [2] Fengxiang Cheng, Haoxuan Li, Fenrong Liu, Robert van Rooij, Kun Zhang, and Zhouchen Lin. Empowering llms with logical reasoning: A comprehensive survey, 2025.
- [3] Zhijiang Guo, Michael Schlichtkrull, and Andreas Vlachos. A survey on automated fact-checking, 2022.
- [4] Zhijing Jin, Abhinav Lalwani, Tejas Vaidhya, Xiaoyu Shen, Yiwen Ding, Zhiheng Lyu, Mrinmaya Sachan, Rada Mihalcea, and Bernhard Schölkopf. Logical fallacy detection, 2022.
- [5] Abhinav Lalwani, Tasha Kim, Lovish Chopra, Christopher Hahn, Zhijing Jin, and Mrinmaya Sachan. Autoformalizing natural language to first-order logic: A case study in logical fallacy detection, 2025.
- [6] Samuel Marks and Max Tegmark. The geometry of truth: Emergent linear structure in large language model representations of true/false datasets, 2024.
- [7] Jidong Tian, Yitian Li, Wenqing Chen, Liqiang Xiao, Hao He, and Yaohui Jin. Diagnosing the first-order logical reasoning ability through LogicNLI. In Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih, editors, *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 3738–3747, Online and Punta Cana, Dominican Republic, November 2021. Association for Computational Linguistics.
- [8] Zhun Yang, Adam Ishay, and Joohyung Lee. Coupling large language models with logic programming for robust and general reasoning from text, 2023.